



پروژه پایانی درس: سیستم‌های خبره (مهندسی مدل‌های زبانی بزرگ)

مدرسین:

دکتر سعید رضا خردپیشه

میلاد وزان

نیم‌سال اول ۱۴۰۵-۱۴۰۶

ارزش: ۹ نمره

راهنمای انتخاب موضوع پروژه:

دانشجویان محترم می‌توانند یکی از موضوعات ارائه‌شده در فهرست این مستند را انتخاب کرده و پروژه خود را بر اساس آن تعریف کنند. همچنین امکان پیشنهاد موضوع جدید توسط دانشجو وجود دارد که در این صورت، سطح‌بندی موضوع توسط مدرس انجام خواهد شد.

سطح‌بندی پروژه‌ها بر اساس تعداد اعضای تیم:

سطح پروژه	تعداد اعضای تیم	توضیحات
سطح ۱	۱ نفر	پروژه‌های فردی
سطح ۲	۲ نفر	پروژه‌های دو نفره
سطح ۳	۳ نفر	پروژه‌های سه نفره

تذکر مهم:

دانشجو با تیم یک‌نفره نیز مجاز به انجام پروژه‌های سطح ۲ و ۳ می‌باشد. اما تیم‌های چندنفره نمی‌توانند پروژه‌های سطح پایین‌تر (مثلاً سطح ۱ با دو نفر) را انتخاب کنند.

تذکرات نهایی:

- انتخاب موضوع باید تا تاریخ ۱ آبان ۱۴۰۵ به مدرسین درس اعلام شود.
 - گزارش نهایی باید شامل: طراحی، پیاده‌سازی، ارزیابی، مستندسازی کامل و خروجی‌های مورد انتظار باشد.
 - پروژه‌ها باید در جلسه پایانی ارائه شوند.
 - تیم‌های چندنفره موظف هستند همه اعضا به‌طور فعال و مؤثر در تمام مراحل پروژه (طراحی، پیاده‌سازی، ارزیابی، مستندسازی و ارائه) مشارکت داشته باشند.
 - در پایان ترم، هر عضو تیم باید گزارش عملکرد فردی خود را به‌صورت جداگانه تحویل دهد و مشخص کند که دقیقاً در کدام بخش‌ها نقش داشته است.
 - هر موضوع از فهرست موضوعات ارائه‌شده (یا پیشنهاد شده توسط دانشجو)، صرف‌نظر از سطح (۱، ۲ یا ۳) و صرف‌نظر از تعداد اعضای تیم، حداکثر توسط ۲ تیم قابل انتخاب می‌باشد.
- توصیه اکید:** تقسیم کار شفاف و استفاده از ابزارهای مدیریت پروژه (مثل ¹GitHub Projects) برای ثبت وظایف و پیشرفت کار، به‌شدت توصیه می‌شود تا از بروز هرگونه ابهام یا اختلاف جلوگیری شود.

¹ https://github.blog/developer-skills/github/github-for-beginners-getting-started-with-github-issues-and-projects/?utm_source=youtube-episode-1&utm_medium=social&utm_campaign=gfb-s3-2026

فهرست موضوعات

.....	۳	سطح ۱
.....	۳	موضوع ۱: سیستم عامل‌های خودمختار با قابلیت "شک" و "بازنگری"
.....	۴	موضوع ۲: دوقلوئی دیجیتال با شخصیت‌پردازی عمیق
.....	۵	موضوع ۳: مرورگر خودکار ادبیات علمی
.....	۶	سطح ۲
.....	۶	موضوع ۱: سیستم تحلیل داده صنعتی با عامل‌های هوشمند
.....	۷	موضوع ۲: عامل هوشمند برای استخراج دانش از مقالات علمی و ساخت گراف دانش
.....	۸	موضوع ۳: عامل تحلیل مکالمات پشتیبانی مشتری
.....	۸	موضوع ۴: شبیه‌سازی جوامع داستانی چندعامله
.....	۱۰	سطح ۳
.....	۱۰	موضوع ۱: عامل خودکار نوشتن مقاله علمی با تحقیق و استناددهی

سطح ۱

موضوع ۱: سیستم عامل‌های خودمختار با قابلیت "شک" و "بازنگری"

چالش اصلی:

سیستم‌های فعلی یا بیش‌ازحد به خود مطمئن هستند (توهم) یا اصلاً اعتماد به نفس ندارند. شما باید سیستمی طراحی کنید که:

- تشخیص دهد کی اشتباه کرده (بدون نیاز به بازخورد خارجی)
- بتواند مسیر استدلال خود را بازنگری کند
- اعتراف به اشتباه کند و مسیر جدیدی برود
- تغییر نظر بدهد اگر شواهد جدیدی پیدا کند

قابلیت‌های پیشنهادی:

۱. ماژول "شک منطقی" (Logical Doubt Module):

- بعد از هر پاسخ، یک ارزیاب داخلی (Internal Critic) پاسخ را از ۳ زاویه مختلف نقد کند:
 - تناقض‌گویی (با پاسخ‌های قبلی)
 - عدم تطابق با دانش پایه
 - ضعف در استدلال (Gap در زنجیره منطق)
- اگر Critic امتیاز زیر ۷۰٪ داد، Agent وارد حالت بازنگری شود.

۲. حالت بازنگری (Reconsideration Mode):

- Agent پاسخ قبلی را کاملاً فراموش کند (Reset Context)
- از یک روش کاملاً متفاوت (مثلاً اگر قبلاً CoT بود، حالا ToT) استفاده کند
- پاسخ جدید را با پاسخ قبلی مقایسه کند و بهترین را انتخاب کند

۳. حافظه "اشتباهات" (Error Memory):

- تمام اشتباهات را با جزئیات ذخیره کند:
 - سوال چه بود؟
 - چه اشتباهی کرد؟
 - دلیل اشتباه چه بود؟ (داده ناکافی، استدلال غلط، سوگیری)
- در مواجهه با سوالات مشابه، هشدار بدهد: "من قبلاً در این موضوع اشتباه کردم، بیایید دقیق‌تر باشیم"

۴. معیار "اعتماد پویا" (Dynamic Confidence):

- هر پاسخ یک امتیاز اطمینان داشته باشد (۰ تا ۱۰۰)
- اگر امتیاز زیر ۵۰ بود، پاسخ را با کلماتی مثل "مطمئن نیستم اما..." شروع کند
- اگر امتیاز بالای ۹۰ بود، پاسخ را با اطمینان کامل بدهد

۵. شبیه‌سازی "دیگری" (Devil's Advocate):

- یک عامل مخالف (Adversary Agent) ایجاد کنید که همیشه سعی کند پاسخ را نقض کند
- Agent اصلی باید بتواند در برابر نقدها دفاع کند یا تغییر نظر بدهد
- این فرآیند باعث می‌شود Agent به نقاط کور خود پی ببرد

خروجی مورد انتظار:

- سیستمی که در ۵۰ سوال چالش برانگیز، در ۲۰٪ موارد "مطمئن نیستم" بگوید (به جای توهم)
- لاگ کامل از فرآیند شک و بازنگری برای هر سوال
- نمودار مقایسه‌ای: دقت سیستم با و بدون مازول شک
- گزارش تحلیلی از الگوهای خطا در سیستم (چه نوع سوالاتی بیشتر باعث اشتباه می‌شوند؟)

موضوع ۲: دوقلوی دیجیتال با شخصیت‌پردازی عمیق

چالش اصلی:

یک دوقلوی دیجیتال از یک شخصیت واقعی یا خیالی بسازید که نه تنها اطلاعات او را داشته باشد، بلکه سبک فکری، لحن، تعصبات، و حتی تکامل شخصیت او را در طول زمان شبیه‌سازی کند. قابلیت‌های پیشنهادی:

۱. استخراج شخصیت از متن (Persona Extraction):
 - از یک مجموعه داده (مثل نامه‌ها، مصاحبه‌ها، کتاب‌ها) شخصیت را استخراج کنید:
 - سبک زبانی: طول جملات، کلمات خاص، ساختار گرامری
 - جهان‌بینی: نگرش به زندگی، سیاست، علم
 - تعصبات: سوگیری‌های شناختی (مثل خوش‌بینی، بدبینی، محافظه‌کاری)
 - دانش: چه چیزهایی می‌داند و چه چیزهایی نمی‌داند؟
۲. تولید پاسخ با شخصیت (Persona-Consistent Response):
 - هر پاسخ باید با شخصیت همخوان باشد
 - اگر از شخصیت خارج شد، یک Critic آن را تشخیص دهد و اصلاح کند
۳. تکامل شخصیت در طول زمان (Persona Evolution):
 - شخصیت ثابت نیست! باید بر اساس تجربیات جدید تکامل یابد
 - مثلاً اگر شخصیت با ایده‌های جدید روبرو شود، ممکن است نظر خود را تغییر دهد
 - مکانیسم تغییر نظر (Opinion Change Mechanism) پیاده‌سازی کنید
۴. حالت "اگر..." (Counterfactual Simulation):
 - "اگر این شخصیت در شرایط X قرار می‌گرفت، چه می‌کرد؟"
 - شبیه‌سازی تصمیمات شخصیت در سناریوهای مختلف
۵. حافظه "عاطفی" (Emotional Memory):
 - هر رویداد یک بار عاطفی (مثبت/منفی) دارد
 - رویدادهای با بار عاطفی بالا بیشتر در حافظه باقی می‌مانند
 - این باعث می‌شود شخصیت انسانی‌تر رفتار کند
۶. تشخیص "ناهماهنگی شناختی" (Cognitive Dissonance Detection):
 - اگر شخصیت دو باور متناقض داشته باشد، سیستم تشخیص دهد و تعارض را نشان دهد
 - شخصیت تلاش کند تعارض را حل کند (تغییر باور یا توجیه)

خروجی مورد انتظار:

- دوقلوی دیجیتال یک شخصیت تاریخی (مثل آلبرت انیشتین یا سقراط) یا یک شخصیت تخیلی
- مصاحبه ۳۰ سوالی با این شخصیت

- مقایسه پاسخ‌های شخصیت در ابتدا و انتهای تعامل (تکامل شخصیت)
- تحلیل روان‌شناختی از شخصیت تولید شده
- گزارش: چگونه می‌توان از این تکنیک برای شبیه‌سازی مشتریان در بازاریابی استفاده کرد؟

موضوع ۳: مرورگر خودکار ادبیات علمی

چالش اصلی:

دانشجویان و محققان برای مرور ادبیات یک حوزه، زمان زیادی صرف جستجو، غربالگری، و مطالعه مقاله‌ها می‌کنند. شما باید سیستم چندعامله‌ای بسازید که:

- یک سوال پژوهشی را به زبان طبیعی دریافت کند
- کوئری‌های جستجوی مناسب برای arXiv تولید کند
- مقاله‌ها را بازاریابی و از نظر مرتبط بودن امتیازدهی کند
- از مقاله‌های مرتبط، بخش‌های روش‌شناسی، یافته‌ها، و کارهای آینده را استخراج کند
- یک گزارش ترکیبی از تم‌های مشترک و شکاف‌های تحقیقاتی تولید کند

معماری پیشنهادی (۶ عامل):

- عامل ساخت کوئری: سوال پژوهشی را به چند کوئری جستجوی arXiv تبدیل کند
 - عامل جستجو: کوئری‌ها را اجرا و نتایج را جمع‌آوری کند
 - عامل غربالگری: هر مقاله را از نظر ارتباط با سوال پژوهشی امتیازدهی کند
 - عامل جمع‌آوری دانش: مقاله‌های مرتبط را دانلود و متن کامل را استخراج کند
 - عامل تحلیل: از هر مقاله، روش‌شناسی، یافته‌ها، و کارهای آینده را استخراج کند
 - عامل ترکیب: تم‌های مشترک و شکاف‌های تحقیقاتی را شناسایی و گزارش نهایی را بنویسد
- خروجی: یک گزارش جامع شامل خلاصه مقاله‌ها، تم‌های اصلی، و پیشنهادات برای تحقیقات آینده

سطح ۲

موضوع ۱: سیستم تحلیل داده صنعتی با عامل‌های هوشمند

الهام‌گرفته از پژوهش: ²Agentic Data Analysis for Intelligent Manufacturing
چالش اصلی:

تجزیه و تحلیل داده در صنایع تولیدی به متخصصان فنی نیاز دارد که هزینه‌بر و کمیاب هستند. کاربران غیرفنی (مدیران تولید، کنترل‌کنندگان کیفیت) باید بتوانند از داده‌ها سوال بپرسند و پاسخ بگیرند. شما باید سیستمی بسازید که:

- کاربران غیرفنی بتوانند با زبان طبیعی از داده‌های صنعتی سوال بپرسند
- داده‌ها را تحلیل کرده و نمودار تولید کند
- پاسخ‌ها را تفسیر کند و به زبان ساده توضیح دهد
- با دقت بالا و قابل تفسیر عمل کند
- برای ۲ دی‌تاست صنعتی مختلف کار کند

قابلیت‌های پیشنهادی:

۱. معماری عامل محور سبک (Lightweight Agentic Framework):

- از LangGraph برای ارکستراسیون عامل‌ها استفاده کنید
- عامل‌ها باید مازولار باشند تا بتوان ابزارهای جدید اضافه کرد

۲. عامل‌های تخصصی:

- عامل درک سوال (Query Understanding Agent): سوال کاربر را به عملیات داده‌ای تبدیل کند
- عامل جستجو (Search Agent): داده‌های مرتبط را پیدا کند
- عامل تحلیل (Analysis Agent): تحلیل آماری و محاسبات را انجام دهد
- عامل تجسم (Visualization Agent): نمودارهای مناسب تولید کند
- عامل تفسیر (Interpretation Agent): نتایج را به زبان ساده توضیح دهد

۳. ارزیابی با دی‌تاست‌های صنعتی:

- از ۲ دی‌تاست تولیدی (مثل داده‌های خط تولید و داده‌های کنترل کیفیت) استفاده کنید
- یک مجموعه سوال استاندارد طراحی کنید
- دقت سیستم را با روش پرامپت مستقیم (بدون عامل) مقایسه کنید

۴. قابلیت توضیح‌دهی (Explainability):

- هر پاسخ باید با گام‌های تحلیل همراه باشد
- کاربر ببیند که سیستم چه محاسباتی انجام داده و چرا به این نتیجه رسیده است

۵. یکپارچه‌سازی با ابزارهای موجود:

- سیستم باید با Pandas، Matplotlib و Seaborn کار کند
- داده‌های CSV یا Excel را به‌عنوان ورودی قبول کند

خروجی مورد انتظار:

- یک سیستم تحلیل داده صنعتی با ۵ عامل تخصصی

² <https://orbit.dtu.dk/en/publications/agentic-data-analysis-for-intelligent-manufacturing-benchmark-dri/>

- اجرا روی ۲ دیتاست صنعتی با ۵۰ سوال استاندارد
- گزارش: سیستم عامل محور در مقابل پرامپت مستقیم چقدر بهتر عمل می‌کند؟
- نمونه‌های عملی از سوالات و پاسخ‌ها

موضوع ۲: عامل هوشمند برای استخراج دانش از مقالات علمی و ساخت گراف دانش

چالش اصلی:

مقالات علمی سرشار از اطلاعات ارزشمند هستند (مفاهیم، روابط بین مفاهیم، روش‌ها، یافته‌ها) اما استخراج و سازماندهی این اطلاعات به صورت دستی تقریباً غیرممکن است. شما باید یک عامل بسازید که:

- یک مقاله علمی را دریافت کند
- مفاهیم کلیدی را استخراج کند
- روابط بین مفاهیم را تشخیص دهد (مثل «X باعث Y می‌شود»، «X یک روش برای Y است»)
- یک گراف دانش از مقاله بسازد
- گراف را به صورت بصری نمایش دهد

قابلیت‌های پیشنهادی:

۱. عامل استخراج مفاهیم (Concept Extraction Agent):

- با استفاده از NER (تشخیص موجودیت‌های نام‌دار) و تکنیک‌های استخراج عبارت کلیدی، مفاهیم اصلی را پیدا کند

- مفاهیم را به دسته‌های «موضوع»، «روش»، «یافته»، «چالش» تقسیم کند

۲. عامل تشخیص روابط (Relation Extraction Agent):

- بین هر جفت مفهوم، نوع رابطه را تشخیص دهد:
 - Causes: X باعث Y می‌شود
 - Part-Of: X بخشی از Y است
 - Method-For: X روشی برای Y است
 - Improves: X بهبود می‌دهد Y را
 - Challenges: X چالشی برای Y است
- از Few-shot Learning با مثال‌های روابط استاندارد استفاده کنید

۳. عامل ساخت گراف (Graph Builder Agent):

- مفاهیم را به عنوان گره و روابط را به عنوان یال ذخیره کند
- گراف را با NetworkX یا Neo4j بسازد
- گراف را به صورت بصری نمایش دهد

۴. عامل خلاصه‌ساز گراف (Graph Summarizer):

- یک خلاصه متنی از گراف تولید کند:
 - مفاهیم اصلی
 - مهم‌ترین روابط
 - خوشه‌های مفهومی (چه مفاهیمی با هم مرتبط‌اند؟)

۵. حالت «مقایسه مقالات» (Paper Comparison Mode):

- کاربر بتواند دو مقاله را با هم مقایسه کند
 - سیستم دو گراف دانش بسازد و شباهت‌ها/تفاوت‌ها را شناسایی کند
- خروجی مورد انتظار:

- یک سیستم استخراج گراف دانش از مقالات علمی (با LangChain و چندین Agent)
- اجرا روی ۲۰ مقاله از حوزه‌های مختلف (مثل پزشکی، کامپیوتر، فیزیک)
- ۱۰ گراف دانش بصری با تحلیل کامل

موضوع ۳: عامل تحلیل مکالمات پشتیبانی مشتری

چالش اصلی:

شرکت‌ها حجم عظیمی از مکالمات پشتیبانی مشتری دارند که تحلیل دستی آن‌ها غیرممکن است. شما باید یک سیستم چندعامله بسازید که:

- مکالمات (متن چت یا ترنسکریپت تماس) را دریافت کند
 - هر مکالمه را خلاصه کند
 - احساسات کلی (مثبت/خنثی/منفی) را تشخیص دهد
 - موضوعات کلیدی مطرح‌شده را استخراج کند
 - یک داشبورد تحلیلی از الگوهای مکالمات تولید کند
- معماری پیشنهادی (۴ عامل):

- عامل خلاصه‌ساز: هر مکالمه را به ۵-۳ جمله کلیدی خلاصه کند
 - عامل احساسات: لحن کلی مکالمه را تشخیص دهد (مثبت/خنثی/منفی)
 - عامل استخراج موضوع: ۳-۲ موضوع اصلی مطرح‌شده را شناسایی کند
 - عامل ترکیب‌کننده: خروجی همه عامل‌ها را در یک ساختار JSON منسجم جمع‌آوری کند
- خروجی: یک فایل JSON شامل خلاصه، احساسات، و موضوعات برای هر مکالمه + یک داشبورد Streamlit

موضوع ۴: شبیه‌سازی جوامع داستانی چندعامله

الهام‌گرفته از پژوهش: BookWorld³

چالش اصلی:

سیستمی که یک رمان یا داستان را به یک جهان زنده تبدیل کند. شخصیت‌ها به‌عنوان عامل‌های مستقل رفتار کنند، با هم تعامل داشته باشند و داستان‌های جدیدی را خلق کنند که به دنیای اصلی وفادار بماند. اما فراتر از شبیه‌سازی ساده، سیستم باید:

- شخصیت‌ها را با شخصیت‌پردازی عمیق از کتاب استخراج کند
- روابط بین شخصیت‌ها را شناسایی و شبیه‌سازی کند
- جغرافیا و محدودیت‌های دنیای داستان را رعایت کند
- داستان‌های جدید و خلاقانه در همان جهان تولید کند

³ <https://aclanthology.org/2025.acl-long.773/>
<https://github.com/alienet1109/BookWorld>

- کاربر بتواند به عنوان یک شخصیت جدید وارد داستان شود
- قابلیت‌های پیشنهادی:

۱. استخراج جهان داستان (World Extraction):

- از یک رمان (مثل هری پاتر، تالکین، یا داستان‌های ایرانی)، استخراج کنید:
- شخصیت‌ها: ویژگی‌ها، اهداف، روابط
- قوانین جهان: جادو، تکنولوژی، محدودیت‌های فیزیکی
- جغرافیا: مکان‌ها، فاصله‌ها، مسیرها
- تاریخچه: رویدادهای مهم قبل از داستان

۲. شخصیت‌پردازی پویا (Dynamic Persona):

- هر شخصیت باید بر اساس تجربیات جدید تکامل یابد
- این تغییرات باید با شخصیت اصلی همخوان باشد

۳. شبیه‌سازی جامعه (Society Simulation):

- شخصیت‌ها باید با هم تعامل کنند (مکالمه، همکاری، درگیری)
- هر شخصیت برنامه روزانه داشته باشد
- رویدادهای تصادفی (مثل جنگ، بلایا) بر جامعه تأثیر بگذارد

۴. تولید داستان جدید (New Story Generation):

- کاربر یک جرقه اولیه بدهد (مثل "چه می‌شد اگر هری پاتر به مدرسه‌ای دیگر می‌رفت؟")
- سیستم یک داستان کامل با همان شخصیت‌ها و همان قوانین تولید کند
- داستان باید غیرقابل پیش‌بینی ولی منطقی باشد

۵. حالت "چه می‌شد اگر..." (Counterfactual Mode):

- "چه می‌شد اگر شخصیت X در رویداد Y تصمیم دیگری می‌گرفت؟"
- سیستم شاخه‌های جایگزین داستان را شبیه‌سازی کند

۶. تعامل کاربر (User Interaction):

- کاربر بتواند به عنوان یک شخصیت جدید وارد جهان شود
- تصمیمات کاربر بر روند داستان تأثیر بگذارد
- شخصیت‌های اصلی به کاربر واکنش نشان دهند (بر اساس شخصیت خود)

۷. وفاداری به منبع (Source Fidelity):

- یک ارزیاب (Evaluator) طراحی کنید که هر خروجی را با کتاب اصلی مقایسه کند
- امتیاز وفاداری (۰ تا ۱۰۰) محاسبه شود
- اگر وفاداری زیر ۸۰٪ بود، خروجی رد شود

خروجی مورد انتظار:

- یک جهان شبیه‌سازی‌شده از یک رمان معروف (پیشنهاد: هری پاتر، ارباب حلقه‌ها، یا شاهنامه)
- حداقل ۵ داستان جدید تولید شده در همان جهان (با وقایع جدید)
- گزارش تحلیلی از تکامل شخصیت‌ها در طول داستان‌های جدید
- مقایسه با روش‌های سنتی (تولید داستان با یک LLM معمولی)

سطح ۳

موضوع ۱: عامل خودکار نوشتن مقاله علمی با تحقیق و استناددهی

چالش اصلی:

نوشتن یک مقاله علمی نیاز به تحقیق، مطالعه منابع، و استناددهی دقیق دارد. شما باید عاملی بسازید که:

- یک موضوع علمی را دریافت کند
- مقالات مرتبط را از Google Scholar یا arXiv پیدا کند
- خلاصه‌ای از هر مقاله بنویسد
- یک پیش‌نویس مقاله با ساختار استاندارد (مقدمه، روش‌شناسی، نتایج، بحث، نتیجه‌گیری) تولید کند
- استنادهای دقیق (با فرمت APA یا IEEE) اضافه کند

قابلیت‌های پیشنهادی:

۱. عامل جستجو (Search Agent):

- با استفاده از Serper API (جستجوی گوگل) یا arXiv API، مقالات مرتبط را پیدا کند
- عنوان، چکیده، نویسندگان، و سال انتشار را استخراج کند
- مقاله‌ها را بر اساس تازگی و تعداد استناد رتبه‌بندی کند

۲. عامل مطالعه (Study Agent):

- خلاصه‌ای از هر مقاله (۲۰۰-۳۰۰ کلمه) با تأکید بر: سوال تحقیق، روش، یافته‌ها
- استخراج کلمات کلیدی و مفاهیم مهم
- شناسایی شکاف‌های تحقیقاتی (Gaps)

۳. عامل نویسنده (Writer Agent):

- بر اساس خلاصه‌ها و ساختار استاندارد، مقاله را بنویسد
- هر بخش باید با منابع پشتیبانی شود
- زبان علمی و لحن مناسب (رسمی، دقیق، بی‌طرفانه)

۴. عامل ویراستار (Editor Agent):

- مقاله را از نظر دقت علمی، انسجام، و گرامر بررسی کند
- پیشنهادات بهبود ارائه دهد
- استنادها را تأیید کند (آیا همه منابع در بخش مراجع وجود دارند؟)

۵. خروجی نهایی:

- یک فایل LaTeX یا Word با مقاله کامل
- فهرست منابع (References)
- یک خلاصه اجرایی (Executive Summary) ۲۰۰ کلمه‌ای

خروجی مورد انتظار:

- یک عامل نوشتن مقاله علمی با ۴ زیر-عامل (جستجو، مطالعه، نویسنده، ویراستار)
- تولید ۵ مقاله در حوزه‌های مختلف (مثلاً یادگیری ماشین، پزشکی، اقتصاد، علوم اجتماعی)
- گزارش: سیستم در چه حوزه‌هایی بهتر عمل می‌کند؟